

Guía de Espaciado en Tailwind CSS

Todas las clases para manejar padding, margin, gap y espaciado entre elementos — con ejemplos y tabla de la escala numérica.

Contenido

- 1. La escala de espaciado (números → tamaños reales)
- 2. Padding — espacio interno
- 3. Margin — espacio externo
- 4. Margin automático (centrar elementos)
- 5. Gap — espacio entre elementos en Flex/Grid
- 6. Space Between — alternativa a gap
- 7. Márgenes y gaps negativos
- 8. Valores responsive (sm, md, lg, xl)
- 9. Valores arbitrarios (personalizados)
- 10. Tabla resumen rápida
- 11. Shrink, Grow y Basis (flex)
- 12. Min-width, Max-width, Min-height, Max-height
- 13. Resumen — shrink/grow/min-max
- 14. Overflow — scroll y contenido cortado
- 15. Familia completa de min-w
- 16. Familia completa de max-w
- 17. Familia de width (w-, incluye w-fit)
- 18. Justify-* e Items-* — alineación en Flex

1. La escala de espaciado

Tailwind no usa píxeles directamente en la clase: usa números que representan múltiplos de **0.25rem** (4px). Esto aplica igual para padding, margin y gap — por eso conviene aprenderla una sola vez.

Fórmula: **número × 4px = tamaño real** (con la fuente por defecto del navegador, 1rem = 16px).

Número	Píxeles	Rem
0	0px	0rem
px	1px	1px
0.5	2px	0.125rem
1	4px	0.25rem
1.5	6px	0.375rem
2	8px	0.5rem
2.5	10px	0.625rem
3	12px	0.75rem
4	16px	1rem
5	20px	1.25rem
6	24px	1.5rem
8	32px	2rem
10	40px	2.5rem
12	48px	3rem
16	64px	4rem
20	80px	5rem
24	96px	6rem
32	128px	8rem
40	160px	10rem
48	192px	12rem
56	224px	14rem
64	256px	16rem
72	288px	18rem

Número	Píxeles	Rem
96	384px	24rem

Ejemplo: **mx-72** que usaste en tu código = $72 \times 4\text{px} = \mathbf{288\text{px de margen a cada lado}}$. En pantallas chicas eso deja casi sin espacio al contenido — de ahí el problema que tuviste.

2. Padding — espacio interno

El padding empuja el **contenido hacia adentro** del elemento, alejándolo de su propio borde. El fondo (background) y el borde del elemento **SÍ** cubren el padding.

Clase	Qué hace
<code>p-4</code>	Padding en los 4 lados
<code>px-4</code>	Padding horizontal (izquierda + derecha)
<code>py-4</code>	Padding vertical (arriba + abajo)
<code>pt-4</code>	Padding solo arriba (top)
<code>pr-4</code>	Padding solo derecha (right)
<code>pb-4</code>	Padding solo abajo (bottom)
<code>pl-4</code>	Padding solo izquierda (left)
<code>ps-4</code>	Padding al inicio (start) — respeta dirección del idioma
<code>pe-4</code>	Padding al final (end) — respeta dirección del idioma

Ejemplo

```
<div class="p-6 bg-white">
  <!-- 24px de padding en los 4 lados -->
  Contenido
</div>

<div class="px-8 py-3 bg-blue-500">
  <!-- 32px a los costados, 12px arriba/abajo -->
  Botón
</div>
```

Tip práctico

Para botones normalmente combinás **px** e **py** por separado en vez de un solo **p**, porque casi siempre necesitás más espacio horizontal que vertical (ej: **px-4 py-2**).

3. Margin — espacio externo

El margin empuja a los **elementos vecinos** alejándolos del elemento actual. A diferencia del padding, el margin es "invisible": no tiene fondo ni color, es puro espacio.

Clase	Qué hace
<code>m-4</code>	Margin en los 4 lados
<code>mx-4</code>	Margin horizontal (izquierda + derecha)
<code>my-4</code>	Margin vertical (arriba + abajo)
<code>mt-4</code>	Margin solo arriba
<code>mr-4</code>	Margin solo derecha
<code>mb-4</code>	Margin solo abajo
<code>ml-4</code>	Margin solo izquierda
<code>ms-4</code>	Margin al inicio (start)
<code>me-4</code>	Margin al final (end)

Eiemplo

```
<div class="mb-4">Tarjeta 1</div>
<div class="mb-4">Tarjeta 2</div>
<!-- Cada tarjeta empuja 16px de espacio hacia abajo -->
```

Padding vs Margin — la diferencia clave

Padding: espacio DENTRO del borde del elemento (entre el borde y su contenido).

Margin: espacio FUERA del borde del elemento (entre el elemento y sus vecinos).

4. Margin automático — centrar elementos

El valor **auto** hace que el navegador reparta el espacio disponible automáticamente. Es la forma clásica de centrar un bloque horizontalmente.

Clase	Qué hace
<code>mx-auto</code>	Centra horizontalmente (el elemento necesita un ancho definido)
<code>my-auto</code>	Centra verticalmente dentro de un contenedor flex
<code>ml-auto</code>	Empuja el elemento hacia la derecha (todo el espacio sobrante a la izquierda)
<code>mr-auto</code>	Empuja el elemento hacia la izquierda (todo el espacio sobrante a la derecha)

Ejemplo

```
<div class="w-96 mx-auto bg-white">
  <!-- Se centra horizontalmente en la página -->
  Contenido centrado
</div>

<div class="flex justify-between">
  <p>Izquierda</p>
  <p class="ml-auto">Empujado a la derecha</p>
</div>
```

5. Gap — espacio entre elementos en Flex/Grid

gap es la forma moderna y más prolija de separar elementos dentro de un contenedor **flex** o **grid**. A diferencia del **margin**, el **gap** solo pone espacio **entre** los elementos — no agrega espacio extra antes del primero ni después del último.

Clase	Qué hace
<code>gap-4</code>	Espacio en ambas direcciones (filas y columnas)
<code>gap-x-4</code>	Espacio horizontal entre columnas
<code>gap-y-4</code>	Espacio vertical entre filas

Eiemplo en Flex

```
<div class="flex gap-4">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>
<!-- 16px de separación entre cada elemento, nada antes ni después -->
```

Eiemplo en Grid

```
<div class="grid grid-cols-3 gap-x-6 gap-y-2">
  <div>1</div><div>2</div><div>3</div>
  <div>4</div><div>5</div><div>6</div>
</div>
<!-- 24px entre columnas, 8px entre filas -->
```

¿Gap o margin en cada elemento?

Preferí **gap** sobre poner **mr-4** a cada elemento hijo. Con **margin** individual el último elemento queda con margen de más (hay que sacárselo a mano con **last:mr-0**); con **gap** eso no pasa nunca.

6. Space Between — alternativa a gap

Antes de que **gap** funcionara bien en Flexbox, se usaba **space-x** / **space-y**. Le agrega margin automáticamente a todos los elementos hijos, excepto al primero.

Clase	Qué hace
<code>space-x-4</code>	Espacio horizontal entre hijos (en fila)
<code>space-y-4</code>	Espacio vertical entre hijos (en columna)
<code>space-x-reverse</code>	Invierte la dirección del espaciado horizontal

```
<div class="flex space-x-4">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>
<!-- Funciona parecido a gap-x-4, pero por margin -->
```

¿Cuál usar hoy?

En proyectos nuevos usá **gap** siempre que puedas — es más simple, más predecible, y funciona igual en flex y grid. **space-x/space-y** lo vas a encontrar sobre todo en código más viejo o tutoriales antiguos.

7. Márgenes negativos

Se usan para que un elemento se "meta" encima de otro o se salga de su contenedor. Se escriben con un guión adelante.

Clase	Qué hace
<code>-m-4</code>	Margin negativo en los 4 lados
<code>-mx-4</code>	Margin negativo horizontal
<code>-mt-4</code>	Margin negativo solo arriba (sube el elemento)
<code>-ml-4</code>	Margin negativo solo izquierda (lo mueve a la izquierda)

```
  
<!-- Sube la imagen 32px, "montándola" sobre el elemento de arriba -->
```

Nota: gap no tiene versión negativa, solo margin.

8. Valores responsive

Cualquier clase de espaciado se puede combinar con un prefijo de tamaño de pantalla, aplicando esa clase a **partir de** ese ancho en adelante (mobile-first).

Prefijo	Se aplica desde
(sin prefijo)	0px en adelante (mobile por defecto)
sm:	640px en adelante
md:	768px en adelante
lg:	1024px en adelante
xl:	1280px en adelante
2xl:	1536px en adelante

Ejemplo — el que corrige tu problema de mx-72

```
<div class="mx-4 sm:mx-12 md:mx-32 lg:mx-72">  
  <!--  
  Mobile: 16px de margen  
  sm:    48px de margen  
  md:    128px de margen  
  lg:    288px de margen (tu valor original)  
  -->  
</div>
```

Se lee de menor a mayor: cada prefijo "pisa" al anterior a partir de ese ancho de pantalla.

9. Valores arbitrarios (personalizados)

Cuando ningún número de la escala te sirve, podés poner un valor exacto entre corchetes []. Sirve para padding, margin y gap por igual.

```
<div class="p-[18px]">...</div>
<div class="mt-[2.5rem]">...</div>
<div class="gap-[10px]">...</div>
<div class="mx-[15%]">...</div>
```

Usalo con moderación: la gracia de Tailwind es la escala consistente. Recurrí a valores arbitrarios solo cuando el diseño realmente lo exige.

10. Tabla resumen rápida

Clases	Para qué sirven
p / px / py / pt / pr / pb / pl	Padding — espacio interno
m / mx / my / mt / mr / mb / ml	Margin — espacio externo
mx-auto / ml-auto / mr-auto	Margin automático — centrar / empujar
gap / gap-x / gap-y	Espacio entre hijos en flex/grid (sin extremos)
space-x / space-y	Espacio entre hijos vía margin (versión antigua)
-m / -mx / -mt...	Margin negativo — superponer elementos
sm: md: lg: xl: 2xl:	Prefijos responsive, se aplican en adelante
p-[18px]	Valor arbitrario / personalizado

Regla general para elegir: usá **padding** para el espacio interno de una tarjeta o botón, **gap** para separar elementos dentro de un flex/grid, y **margin** para separar un bloque entero del resto de la página.

11. Shrink, Grow y Basis — cómo ceden espacio los elementos flex

Estas clases no son espaciado en sí (no agregan huecos), pero definen **cómo se comporta cada elemento cuando sobra o falta espacio** dentro de un flex — por eso van de la mano con min-w y max-w a la hora de resolver problemas de layout.

flex-shrink — permitir o impedir que se achique

Clase	Qué hace
<code>shrink</code>	Default. El elemento SÍ puede achicarse si falta espacio (flex-shrink: 1)
<code>shrink-0</code>	El elemento NUNCA se achica, mantiene su tamaño original
<code>shrink-[2]</code>	Se achica el doble de rápido que uno con shrink normal (valor arbitrario)

```
<div class="flex">
  
  <!-- El logo mantiene su tamaño siempre -->
  <p class="min-w-0 truncate">Texto que se achica y trunca</p>
</div>
```

flex-grow — permitir que ocupe espacio libre

Clase	Qué hace
<code>grow-0</code>	Default. El elemento no crece más de lo que necesita su contenido
<code>grow</code>	El elemento crece para ocupar TODO el espacio libre disponible (flex-grow: 1)
<code>grow-[2]</code>	Crece el doble de rápido que un elemento con grow normal

```
<div class="flex">
  <div>Fijo</div>
  <div class="grow">Ocupa todo el espacio sobrante</div>
  <div>Fijo</div>
</div>
```

flex-basis — tamaño inicial antes de repartir espacio

Clase	Qué hace
<code>basis-0</code>	Arranca desde 0, todo el tamaño final sale de grow/shrink
<code>basis-1/2</code>	Arranca ocupando el 50% del contenedor
<code>basis-64</code>	Arranca con 256px (usa la misma escala que width)

Clase	Qué hace
<code>basis-auto</code>	Default. Arranca con el tamaño natural del contenido

Atajos combinados

Clase	Qué hace
<code>flex-1</code>	Equivale a flex: 1 1 0% — crece, se achica, ignora su tamaño natural
<code>flex-auto</code>	Equivale a flex: 1 1 auto — crece, se achica, respeta su tamaño natural
<code>flex-initial</code>	Equivale a flex: 0 1 auto — se achica pero no crece (default de flex)
<code>flex-none</code>	Equivale a flex: none — no crece ni se achica, tamaño fijo siempre

flex-1 es muy usado para "que este elemento ocupe todo el resto del espacio", por ejemplo un input que crece hasta pegar contra un botón de tamaño fijo.

12. Min-width, Max-width, Min-height, Max-height

Controlan los límites de achicamiento y crecimiento de un elemento, en cualquier tipo de layout (no solo flex). Muy usados junto con shrink/grow para resolver problemas responsive.

min-w — ancho mínimo

Clase	Qué hace
<code>min-w-0</code>	Sin ancho mínimo — permite achicarse todo lo que haga falta (clave con truncate)
<code>min-w-full</code>	Ancho mínimo = 100% del contenedor padre
<code>min-w-min</code>	Ancho mínimo = el mínimo posible según el contenido (min-content)
<code>min-w-max</code>	Ancho mínimo = lo que ocupa el contenido sin cortarse (max-content)
<code>min-w-fit</code>	Ancho mínimo ajustado al contenido (fit-content)
<code>min-w-64</code>	Ancho mínimo fijo de 256px (usa la escala de espaciado)

max-w — ancho máximo

Clase	Qué hace
<code>max-w-none</code>	Sin límite de ancho máximo
<code>max-w-xs / sm / md / lg / xl</code>	Anchos máximos predefinidos (20rem, 24rem, 28rem, 32rem, 36rem...)
<code>max-w-screen-lg</code>	Ancho máximo = el breakpoint lg (1024px)
<code>max-w-full</code>	Ancho máximo = 100% del contenedor padre
<code>max-w-prose</code>	Ancho máximo ideal para lectura de texto (65 caracteres aprox.)
<code>max-w-96</code>	Ancho máximo fijo de 384px

max-w con **mx-auto** es el combo clásico para centrar contenido con un ancho tope, dejando que se achique libremente en pantallas chicas:

```
<div class="max-w-2xl mx-auto px-4">
  <!-- Nunca supera 42rem de ancho, se centra,
        y en mobile se achica hasta el borde con margen -->
</div>
```

min-h y max-h — lo mismo pero en altura

Clase	Qué hace
<code>min-h-0</code>	Sin altura mínima
<code>min-h-screen</code>	Altura mínima = 100vh (toda la pantalla)
<code>min-h-full</code>	Altura mínima = 100% del contenedor padre
<code>max-h-64</code>	Altura máxima fija de 256px (útil con overflow-y-auto para scroll)
<code>max-h-screen</code>	Altura máxima = 100vh

13. Resumen — shrink/grow/min-max

Clase	Para qué sirve
<code>shrink-0</code>	El elemento NO se achica nunca (ideal para imágenes/iconos/botones)
<code>shrink</code>	El elemento SÍ puede achicarse (default)
<code>grow</code>	El elemento ocupa todo el espacio libre disponible
<code>flex-1</code>	Atajo: crece + se achica, ignorando su tamaño natural
<code>min-w-0</code>	Permite achicar por debajo del contenido (clave junto a truncate)
<code>max-w-*</code>	Tope de ancho — combinar con mx-auto para centrar con límite
<code>min-h-screen</code>	Ocupa como mínimo toda la altura de la pantalla

Patrón típico que resolviste en tu tarjeta: imagen con **shrink-0** (que no se toque) + contenedor de texto con **min-w-0** (que ceda espacio) + **truncate** (que corte prolijo).

14. Overflow — qué pasa cuando el contenido no entra

Controla el comportamiento cuando el contenido de un elemento es más grande que su contenedor. Se puede aplicar en las dos direcciones a la vez, o por separado en horizontal (x) y vertical (y).

overflow — ambas direcciones

Clase	Qué hace
<code>overflow-auto</code>	Agrega scroll SOLO si hace falta (el navegador decide)
<code>overflow-hidden</code>	Corta el contenido que se pasa, sin scroll ni barra
<code>overflow-visible</code>	Default. El contenido se puede salir del contenedor, se ve igual
<code>overflow-scroll</code>	Siempre muestra la barra de scroll, aunque no haga falta
<code>overflow-clip</code>	Corta el contenido igual que hidden, pero sin crear scroll container

overflow-x — solo horizontal

Clase	Qué hace
<code>overflow-x-auto</code>	Scroll horizontal solo si el contenido no entra a lo ancho
<code>overflow-x-hidden</code>	Corta el contenido que se pasa horizontalmente
<code>overflow-x-visible</code>	El contenido se puede salir del contenedor a los costados
<code>overflow-x-scroll</code>	Siempre muestra scroll horizontal
<code>overflow-x-clip</code>	Corta el contenido horizontal sin crear scroll container

overflow-y — solo vertical (la que preguntaste)

Clase	Qué hace
<code>overflow-y-auto</code>	Scroll vertical solo si el contenido no entra a lo alto
<code>overflow-y-hidden</code>	Corta el contenido que se pasa verticalmente
<code>overflow-y-visible</code>	El contenido se puede salir del contenedor arriba/abajo
<code>overflow-y-scroll</code>	Siempre muestra scroll vertical
<code>overflow-y-clip</code>	Corta el contenido vertical sin crear scroll container

Ejemplo — lista con scroll vertical

```
<div class="max-h-96 overflow-y-auto">
  <!--
  Si el contenido supera 384px de alto (max-h-96),
  aparece scroll vertical automáticamente.
  Si entra, no se ve ninguna barra.
  -->
  <div>Item 1</div>
  <div>Item 2</div>
  <div>... muchos items más</div>
</div>
```

auto vs scroll — la diferencia clave

overflow-y-auto: la barra de scroll solo aparece si realmente hace falta (comportamiento normal y recomendado en la mayoría de los casos).

overflow-y-scroll: la barra siempre está visible, haga falta o no — útil solo si querés evitar que el layout "salte" cuando el contenido cambia de tamaño.

overscroll-behavior — evitar el scroll en cadena

Bonus relacionado: cuando llegás al final del scroll interno de un elemento, por defecto el scroll sigue empujando a la página completa ("scroll chaining"). Estas clases lo evitan:

Clase	Qué hace
<code>overscroll-auto</code>	Default. El scroll se pasa a la página cuando llega al límite
<code>overscroll-contain</code>	El scroll se frena en el borde, no afecta a la página ni rebota
<code>overscroll-none</code>	Igual que contain, sin efecto rebote (bounce) en el propio elemento
<code>overscroll-y-contain</code>	Igual que contain, pero solo en el eje vertical

```
<div class="max-h-96 overflow-y-auto overscroll-contain">
  <!-- Un modal o dropdown con lista larga:
  al llegar al final del scroll interno,
  no arrastra a toda la página con él -->
</div>
```

15. Familia completa de min-w (min-width)

En la sección 12 vimos los valores especiales de **min-w**. Acá está la familia completa: además de esos valores especiales, min-w también acepta toda la escala numérica (igual que width) y fracciones.

Valores especiales (keywords)

Clase	Qué hace
<code>min-w-0</code>	0px — sin ancho mínimo, se achica todo lo necesario
<code>min-w-px</code>	1px de ancho mínimo
<code>min-w-full</code>	100% del contenedor padre
<code>min-w-min</code>	min-content — el mínimo posible sin cortar palabras/contenido
<code>min-w-max</code>	max-content — lo que ocupa el contenido sin envolver ni cortar
<code>min-w-fit</code>	fit-content — se ajusta al contenido, con límites del contenedor
<code>min-w-screen</code>	100vw — ancho mínimo = ancho completo de la ventana
<code>min-w-none</code>	Sin mínimo (equivale a min-w-0 en la práctica)

Escala numérica (igual que width/spacing)

Clase	Tamaño
<code>min-w-0</code>	0px
<code>min-w-1</code>	4px
<code>min-w-2</code>	8px
<code>min-w-4</code>	16px
<code>min-w-6</code>	24px
<code>min-w-8</code>	32px
<code>min-w-10</code>	40px
<code>min-w-12</code>	48px
<code>min-w-16</code>	64px
<code>min-w-20</code>	80px
<code>min-w-24</code>	96px
<code>min-w-32</code>	128px

Clase	Tamaño
<code>min-w-40</code>	160px
<code>min-w-48</code>	192px
<code>min-w-56</code>	224px
<code>min-w-64</code>	256px
<code>min-w-72</code>	288px
<code>min-w-96</code>	384px

Fracciones (porcentajes)

Clase	Tamaño
<code>min-w-1/2</code>	50%
<code>min-w-1/3</code>	33.33%
<code>min-w-2/3</code>	66.66%
<code>min-w-1/4</code>	25%
<code>min-w-3/4</code>	75%
<code>min-w-1/5</code>	20%
<code>min-w-full</code>	100%

Valor arbitrario

```
<div class="min-w-[250px]">...</div>
<div class="min-w-[30ch]">...</div>
<!-- Cualquier valor CSS válido entre corchetes -->
```

Uso más común de cada uno

min-w-0: liberar un flex/grid item para que se achique (el que resolvió tu problema).

min-w-full: forzar que un elemento ocupe como mínimo todo el ancho del padre.

min-w-fit / min-w-max: evitar que botones o badges se corten o envuelvan el texto.

min-w-[Npx]: cuando necesitas un piso exacto de ancho, por ejemplo en una columna de tabla.

Nota: todo lo mismo aplica en espejo para **max-w** (con las mismas variantes numéricas, fracciones y keywords) y para **min-h / max-h** en el eje vertical.

16. Familia completa de max-w (max-width)

El espejo de min-w. Además de los tamaños predefinidos por nombre (xs, sm, md...), acepta la misma escala numérica, fracciones y valores arbitrarios.

Tamaños predefinidos por nombre (los más usados para contenido)

Clase	Tamaño
<code>max-w-xs</code>	320px
<code>max-w-sm</code>	384px
<code>max-w-md</code>	448px
<code>max-w-lg</code>	512px
<code>max-w-xl</code>	576px
<code>max-w-2xl</code>	672px
<code>max-w-3xl</code>	768px
<code>max-w-4xl</code>	896px ← el que usamos en tu carrito
<code>max-w-5xl</code>	1024px
<code>max-w-6xl</code>	1152px
<code>max-w-7xl</code>	1280px

Ligados a los breakpoints de pantalla

Clase	Tamaño
<code>max-w-screen-sm</code>	640px — igual al breakpoint sm
<code>max-w-screen-md</code>	768px — igual al breakpoint md
<code>max-w-screen-lg</code>	1024px — igual al breakpoint lg
<code>max-w-screen-xl</code>	1280px — igual al breakpoint xl

Valores especiales (keywords)

Clase	Qué hace
<code>max-w-none</code>	Sin límite de ancho máximo
<code>max-w-full</code>	100% del contenedor padre
<code>max-w-min</code>	min-content — el mínimo posible

Clase	Qué hace
<code>max-w-max</code>	max-content — lo que ocupa el contenido
<code>max-w-fit</code>	fit-content — se ajusta al contenido
<code>max-w-prose</code>	65ch aprox. — ancho ideal para bloques de texto largo

Fracciones y valor arbitrario

```
<div class="max-w-1/2">...</div>    <!-- 50% -->
<div class="max-w-[600px]">...</div> <!-- valor exacto -->
```

Cuál elegir: para contenedores de página (como tu carrito) usá los nombrados (xs → 7xl) — son los pensados para eso. Para límites relativos al padre usá full/fit/min/max.

17. Familia de width (w-) — incluye w-fit

El ancho "normal" del elemento (no mínimo ni máximo). Comparte la misma escala numérica que vimos en spacing/min-w, más estos valores especiales:

Clase	Qué hace
w-auto	Default. El navegador calcula el ancho según el contexto
w-full	100% del contenedor padre
w-screen	100vw — ancho completo de la ventana del navegador
w-min	min-content — el mínimo posible sin cortar el contenido
w-max	max-content — el ancho exacto que ocupa el contenido, sin envolver
w-fit	fit-content — se ajusta al contenido, respetando límites del padre

Escala numérica (igual que min-w/spacing)

Clase	Tamaño
w-4	16px
w-8	32px
w-16	64px
w-32	128px
w-48	192px
w-64	256px
w-96	384px

Fracciones

Clase	Tamaño
w-1/2	50%
w-1/3 / w-2/3	33.33% / 66.66%
w-1/4 / w-3/4	25% / 75%
w-1/5 ... w-4/5	20% a 80%, de a quintos
w-1/12 ... w-11/12	Sistema de 12 columnas, de a doceavos

Ejemplo: w-fit para el link "Volver"

```
<a href="#" class="flex items-center gap-1 w-fit">
  <!-- w-fit: el link ocupa SOLO lo que mide su contenido
    (flecha + texto), no todo el ancho disponible.
    Sin esto, el área clickeable se estiraría de más. -->
  
  <span>Volver</span>
</a>
```

Resumen w vs min-w vs max-w

Clases	Comportamiento
<code>w-64</code>	Ancho fijo — nunca cambia, aunque el contenedor tenga más o menos lugar
<code>min-w-0</code>	Piso — puede achicarse hasta ahí, nunca menos
<code>max-w-4xl</code>	Techo — puede crecer hasta ahí, nunca más
<code>min-w-0 + max-w-4xl</code> juntos	Se combinan: fluido entre esos dos límites

18. Justify-* e Items-* — alineación en Flex

No son clases de espaciado (no agregan huecos), pero van de la mano con gap y flex-col/row para resolver "por qué mis elementos no se mueven para donde quiero". La clave: Flexbox tiene un **eje principal** y un **eje cruzado**, y cuál es horizontal o vertical depende de la dirección del contenedor.

La regla que lo explica todo

Dirección	justify-* controla	items-* controla
<code>flex</code> / <code>flex-row</code> (default)	Eje principal = HORIZONTAL	Eje cruzado = VERTICAL
<code>flex-col</code>	Eje principal = VERTICAL	Eje cruzado = HORIZONTAL

justify-* siempre controla el eje principal. **items-*** siempre controla el eje cruzado. Lo que cambia con flex-col es CUÁL de los dos es horizontal o vertical.

justify-* — familia completa

Clase	Qué hace
<code>justify-start</code>	Default. Agrupa todo al inicio del eje principal
<code>justify-end</code>	Agrupa todo al final del eje principal
<code>justify-center</code>	Centra todo en el eje principal
<code>justify-between</code>	Primero al inicio, último al final, resto repartido en el medio
<code>justify-around</code>	Espacio igual alrededor de cada elemento
<code>justify-evenly</code>	Espacio exactamente igual entre todos, incluidos los bordes

items-* — familia completa

Clase	Qué hace
<code>items-start</code>	Agrupa todo al inicio del eje cruzado
<code>items-end</code>	Agrupa todo al final del eje cruzado
<code>items-center</code>	Centra todo en el eje cruzado
<code>items-baseline</code>	Alinea por la línea base del texto (útil con fuentes de distinto tamaño)
<code>items-stretch</code>	Default. Estira los elementos para ocupar todo el eje cruzado

Ejemplo — el caso del carrito

```

<!-- Texto y botón en FILA, uno pegado a cada punta -->
<div class="flex justify-between items-center">
  <p>Total de puntos 2534</p>
  <button>Ir a pagar</button>
</div>

<!-- Texto y botón en COLUMNA, ambos pegados a la derecha -->
<div class="flex flex-col items-end gap-2">
  <p>Total de puntos 2534</p>
  <button>Ir a pagar</button>
</div>

```

Cuándo usar cada uno

Clase	Caso de uso típico
<code>justify-between</code>	Header con logo a la izquierda y menú a la derecha (flex-row)
<code>justify-end</code>	Agrupar varios elementos juntos, todos pegados a un extremo (flex-row)
<code>justify-center</code>	Centrar un botón o un grupo chico dentro de un contenedor ancho
<code>items-center</code>	Alinear verticalmente ícono + texto de distinta altura, en flex-row
<code>items-end</code>	Pegar contenido a la derecha cuando el contenedor es flex-col
<code>items-stretch</code>	Que varias tarjetas en fila tengan siempre la misma altura